

Agentes persistentes · Sub-agentes · Orquestación · Multitasking · Contexto compartido · Pipelines de programación

2

GUÍA 2 — MÓDULO 2

Multi-agente

De un agente a un sistema — cómo trabajan juntos en OpenClaw

I — Cuándo tiene sentido usar multi-agente

La primera pregunta no es cómo configurar múltiples agentes — es si realmente los necesitás. OpenClaw por defecto corre en modo un solo agente por una razón: la mayoría de los casos de uso no requieren más.

Un agente bien configurado puede manejar múltiples canales, usar distintas herramientas y mantener conversaciones complejas en paralelo. La complejidad de multi-agente tiene un costo real: más configuración, más superficie de ataque, más cosas que pueden fallar.

Agregar un segundo agente cuando...	No agregar cuando...
Necesitás aislar permisos: el agente público no puede tener acceso a shell, pero el agente interno sí.	Un solo agente con herramientas configuradas por canal resuelve el problema.
Hay tareas que toman mucho tiempo y bloquean la sesión principal: investigación larga, generación de código, análisis de documentos.	La tarea es corta y el usuario puede esperar la respuesta.
Necesitás especialización real: un agente programador con herramientas de código, uno de atención al cliente sin acceso al sistema.	Es más cómodo pero no cambia el resultado — la complejidad no se justifica.
El contexto 'se contamina': el agente mezcla información de distintos dominios o usuarios.	El problema es de configuración de MEMORY.md, no de arquitectura.

→ **La regla de oro**

Agregá un agente nuevo solo cuando podés nombrar exactamente el problema que resuelve y por qué un solo agente bien configurado no puede resolverlo. Si la respuesta es vaga, no lo agregues todavía.

II — Los dos patrones de multi-agente en OpenClaw

OpenClaw tiene dos formas distintas de correr múltiples agentes. Es importante entender la diferencia antes de configurar cualquier cosa porque resuelven problemas diferentes.

Agentes persistentes — viven siempre

Son agentes completos con su propio workspace, AGENTS.md, SOUL.md, memoria y canales asignados. Corren indefinidamente dentro del mismo Gateway. Cada uno tiene su propia identidad y no sabe nada de los otros por defecto.

```
# Caso típico: agente personal + agente de equipo
# Ambos corren en el mismo VPS, mismo Gateway
# Cada uno con su workspace separado

# Agente personal — acceso total
$ agente-personal:
  workspace: ~/.openclaw/workspace-personal
  canal: WhatsApp personal
  herramientas: shell, browser, email, calendario

# Agente de equipo — acceso restringido
$ agente-equipo:
  workspace: ~/.openclaw/workspace-equipo
  canal: Slack del equipo
  herramientas: lectura de archivos, búsqueda web
```

Sub-agentes — viven por una tarea

Son agentes temporales que el agente principal lanza para ejecutar una tarea en paralelo. Cuando terminan, anuncian el resultado de vuelta al chat del agente principal y se archivan automáticamente. No tienen workspace propio — heredan la configuración del agente que los lanzó.

```
# El agente principal recibe un pedido largo:
# "Investigá los últimos 5 competidores de nuestro producto"

# En lugar de bloquear la sesión principal,
# lanza un sub-agente para hacer la investigación:

$ /sub-agentes lanzar agente-principal "Investigá competidores: Competitor A, B, C, D, E. Resumí fortalezas y debilidades de cada uno."

# El sub-agente trabaja en background
# Cuando termina, el resultado aparece en el chat principal
# La sesión principal queda libre mientras tanto
```

Agentes persistentes

Sub-agentes

¿Cuándo viven?	Siempre — mientras el Gateway esté activo	Solo mientras dura su tarea (auto-archivo a los 60 min)
¿Tienen workspace propio?	Sí — con todos sus archivos .md	No — heredan del agente que los lanzó
¿Tienen memoria propia?	Sí — MEMORY.md independiente	No — contexto solo durante la tarea
¿Para qué sirven?	Especialización, aislamiento de permisos, canales separados	Paralelismo, tareas largas, multitasking
¿Pueden comunicarse con otros?	No directamente — por diseño	Solo con su agente padre

Setup — agentes persistentes en el mismo Gateway

III — Instalar y configurar agentes persistentes

Paso 1 — Crear el workspace del nuevo agente

```
# Crear el directorio del workspace
$ mkdir -p ~/.openclaw/workspace-agente2

# Copiar la estructura base desde el agente principal
$ cp ~/.openclaw/workspace/AGENTS.md ~/.openclaw/workspace-agente2/
$ cp ~/.openclaw/workspace/SOUL.md ~/.openclaw/workspace-agente2/

# Editar los archivos para este agente específico
# AGENTS.md: ajustar herramientas y rutinas
# SOUL.md: ajustar nombre, personalidad y límites
```

Paso 2 — Definir el agente en openclaw.json

```
# En openclaw.json — sección agents.list:
"agentes": {
  "lista": [
    {
      "id": "agente-principal",
      "workspace": "~/.openclaw/workspace",
      "directorioAgente": "~/.openclaw/agentes/principal"
    },
    {
      "id": "agente-equipo",
      "workspace": "~/.openclaw/workspace-equipo",
      "directorioAgente": "~/.openclaw/agentes/equipo",
      "herramientas": {
        "denegar": ["shell", "browser", "email"]
      }
    }
  ]
}
```

Paso 3 — Configurar bindings (enrutamiento)

Los bindings definen qué mensajes van a qué agente. Sin bindings explícitos, OpenClaw no sabe a dónde rutear cada mensaje.

```
# En openclaw.json — sección bindings:
"bindings": [
  {
    "agente": "agente-principal",
    "canal": "whatsapp",
    "cuentaId": "tu-numero-whatsapp"
  },
  {
    "agente": "agente-equipo",
    "canal": "slack",
    "espacioTrabajo": "tu-workspace-slack"
  }
]
$ ]
```

Paso 4 — Verificar la configuración

```
# Listar agentes activos y sus bindings:
$ openclaw agentes listar --bindings

# Verificar que el Gateway levanta sin errores:
$ openclaw iniciar --verificar

# Probar enrutamiento enviando un mensaje de prueba
# desde cada canal y verificando que llega al agente correcto
```

⚠ Aislamiento de sesiones — no es automático

Cada agente tiene su propio directorio de agente (agentDir) con sus sesiones, perfiles de auth y estado. Nunca reutilices el mismo agentDir para dos agentes — causa colisiones de auth y comportamiento impredecible. Si querés compartir credenciales entre agentes, copiá el archivo auth-profiles.json al directorio del nuevo agente.

Sub-agentes — paralelismo y tareas largas

IV — Sub-agentes en profundidad

Los sub-agentes son la herramienta de paralelismo de OpenClaw. Cuando el agente principal tiene una tarea larga o puede dividir trabajo en partes independientes, lanza sub-agentes para ejecutarlas en background sin bloquear la sesión principal.

Cómo funciona el ciclo



Configurar límites de sub-agentes en openclaw.json

```

# En openclaw.json — sección agentes.defaults:
"agentes": {
  "defaults": {
    "subAgentes": {
      "profundidadMaxLanzamiento": 2,
      "maxHijosPorAgente": 5,
      "maxConcurrentes": 8,
      "archivarDespuesDeMinutos": 60,
      "modelo": "claude-haiku-4-5-20251001"
    }
  }
}

# profundidadMaxLanzamiento: 1 = solo el principal puede lanzar sub-agentes
# profundidadMaxLanzamiento: 2 = un sub-agente puede lanzar sus propios hijos (patrón orquestador)
# modelo: usar un modelo más barato para sub-agentes ahorra costos significativamente

```

→ Profundidad 2 — el patrón orquestador

Con `profundidadMaxLanzamiento: 2`, el agente principal lanza un sub-agente orquestador, y ese orquestador puede lanzar varios sub-agentes trabajadores en paralelo. Es el patrón más potente para pipelines complejos. El orquestador recibe automáticamente las herramientas `sessions_spawn`, `sub-agentes`, `sessions_list` y `sessions_history` para coordinar a sus hijos.

V — Comunicación y paso de contexto

La comunicación entre agentes en OpenClaw es el punto que más confunde. Por diseño, los agentes persistentes no se hablan directamente. Los sub-agentes solo se comunican con su agente padre. Esto es intencional — limita la superficie de ataque y hace el sistema predecible.

Cómo pasa contexto un sub-agente a su padre

```
# El agente principal lanza el sub-agente con contexto inicial:
$ /sub-agentes lanzar agente-principal "Analizá el archivo reporte-marzo.md
$ que está en el workspace. Resumí los puntos clave en menos de 200 palabras
$ y devolvé el resultado como lista numerada."

# El sub-agente tiene acceso a:
# - La tarea recibida en el mensaje de lanzamiento
# - Las herramientas habilitadas para sub-agentes
# - El workspace del agente padre (lectura)

# Cuando termina, anuncia el resultado en el chat del padre
# El padre ve el resultado y puede actuar sobre él
```

Comunicación indirecta entre agentes persistentes

Los agentes persistentes no tienen un canal directo entre sí. Los patrones que usa la comunidad para coordinarlos son:

Patrón	Cómo funciona	Cuándo usarlo
Archivo compartido	Un agente escribe un archivo en una ruta compartida. El otro lo lee en su próximo ciclo de heartbeat.	Datos que no son urgentes y pueden esperar el próximo heartbeat
Canal compartido	Ambos agentes tienen acceso de lectura a un canal común (ej: un grupo de Telegram). El agente A publica, el B lee.	Coordinación simple sin necesidad de respuesta inmediata
Skill de mensajería interna	Un skill personalizado que hace una llamada HTTP interna al Gateway para rutear un mensaje a otro agente.	Cuando necesitás respuesta real y coordinación activa — requiere desarrollo
Usuario como intermediario	El usuario recibe el resultado del agente A y se lo pasa al agente B manualmente.	Para flujos poco frecuentes donde la fricción es aceptable

RFC Agent Teams — lo que viene

Hay una propuesta activa en el repositorio de OpenClaw (RFC #10036) para agregar 'Agent Teams': comunicación directa entre sub-agentes, lista de tareas compartida con dependencias y coordinación en tiempo real. Cuando esté disponible, resuelve el problema de raíz. Al momento de esta guía, todavía no está lanzado.



Multitasking — ejecución paralela real

VI — Multitasking con sub-agentes

El caso de uso más potente de los sub-agentes es la ejecución paralela: lanzar múltiples sub-agentes al mismo tiempo para que trabajen en partes independientes de un problema y traigan los resultados juntos.

Ejemplo real — investigación paralela

```
# Tarea: comparar 4 herramientas de automatización
# En lugar de investigarlas una por una (secuencial)
# lanzar 4 sub-agentes en paralelo

$ /sub-agentes lanzar agente-principal "Investigá Make (antes Integromat): precios, limitaciones, casos de uso reales. Devolvé en formato estructurado."
$ /sub-agentes lanzar agente-principal "Investigá Zapier: precios, limitaciones, casos de uso reales. Devolvé en formato estructurado."
$ /sub-agentes lanzar agente-principal "Investigá n8n: precios, limitaciones, casos de uso reales. Devolvé en formato estructurado."
$ /sub-agentes lanzar agente-principal "Investigá Activepieces: precios, limitaciones, casos de uso reales. Devolvé en formato estructurado."

# Los 4 corren en paralelo
# Cada uno anuncia su resultado cuando termina
# El agente principal puede hacer el análisis comparativo
# cuando tiene todos los resultados
```

Límites que importan en la práctica

Límite	Valor por defecto	Qué pasa si se supera
maxConcurrentes	8 sub-agentes activos en simultáneo	Los nuevos lanzamientos quedan en cola hasta que se libere un slot
maxHijosPorAgente	5 hijos por sesión de agente	El agente padre no puede lanzar más hasta que terminen algunos
profundidadMaxLanzamiento	1 (solo el principal puede lanzar)	Con profundidad 2, los orquestadores pueden lanzar trabajadores
archivarDespuesDeMinutos	60 minutos	El sub-agente se archiva aunque no haya terminado — diseñar tareas de menos de 60 min o configurar más tiempo

Casos reales — pipelines de programación

VII — Multi-agente en programación

Los pipelines de desarrollo son el caso de uso más documentado y más potente de multi-agente en OpenClaw. La idea central: separar las responsabilidades de escritura, revisión y testing en agentes especializados que trabajan en secuencia o en paralelo.

Pipeline: código → revisión → testing

Documentado en producción por la comunidad. Un agente programador escribe el código, un revisor lo evalúa, un tester lo prueba. La orquestación determina si el flujo es secuencial (un paso a la vez) o con reintentos.

```
# Configuración de agentes en openclaw.json:
"agentes": {
  "lista": [
    {
      "id": "programador",
      "workspace": "~/openclaw/workspace-programador",
      "herramientas": { "permitir": ["shell", "lectura-archivos", "escritura-archivos"] }
    },
    {
      "id": "revisor",
      "workspace": "~/openclaw/workspace-revisor",
      "herramientas": { "permitir": ["lectura-archivos"] }
    },
    {
      "id": "tester",
      "workspace": "~/openclaw/workspace-tester",
      "herramientas": { "permitir": ["shell", "lectura-archivos"] }
    }
  ]
}
```

El flujo en AGENTS.md del orquestador

```
# AGENTS.md del agente orquestador:

## Flujo de desarrollo
$ Cuando recibo una tarea de programación:
$ 1. Lanzar sub-agente "programador" con la especificación completa
$ 2. Esperar el resultado del programador
$ 3. Lanzar sub-agente "revisor" con el código generado
```

- \$ 4. Si el revisor devuelve problemas, volver al paso 1 (máximo 3 iteraciones)
- \$ 5. Si el revisor aprueba, lanzar sub-agente "tester"
- \$ 6. Si los tests pasan, notificar al usuario con el resultado final
- \$ 7. Si los tests fallan, notificar al usuario y esperar instrucciones

⚠ **El problema del flujo no determinístico**

Cuando la orquestación la decide el LLM (el agente elige cuándo lanzar el revisor, cuándo reintentar), el flujo puede ser impredecible. El modelo puede saltarse pasos, reintentar de más o decidir que algo está listo cuando no lo está.

Para flujos críticos donde necesitás garantías sobre el orden de ejecución, la comunidad recomienda usar Lobster (el motor de workflows de OpenClaw) en lugar de instrucciones en AGENTS.md. Lobster permite definir el flujo en YAML con lógica determinística — el LLM hace el trabajo creativo, el YAML maneja la secuencia.

Otros casos documentados

Pipeline	Agentes involucrados	Qué hace cada uno
Research automatizado	Buscador + Sintetizador + Formateador	Buscador recolecta fuentes en paralelo. Sintetizador extrae lo relevante. Formateador produce el documento final.
Revisión de PRs	Revisor de código + Revisor de seguridad + Generador de resumen	Cada revisor analiza una dimensión diferente del PR. El orquestador consolida los comentarios.
Generación de contenido	Investigador + Redactor + Editor	Investigador recolecta datos. Redactor escribe el borrador. Editor ajusta tono y estilo.
Procesamiento de datos	Extractor + Validador + Transformador + Cargador	Cada etapa del pipeline ETL en un agente especializado con acceso solo a lo que necesita.

VIII — Costos en sistemas multi-agente

Cada sub-agente tiene su propio contexto y consume tokens independientemente. Un sistema mal diseñado puede multiplicar los costos por el número de agentes. Hay tres palancas concretas para controlarlo.

Estrategia	Implementación	Ahorro estimado
Modelo diferente para sub-agentes	Configurar <code>agentes.defaults.subAgentes.modelo</code> con un modelo más barato (ej: Haiku en lugar de Opus)	40-60% según documentación de la comunidad
Instrucciones concisas al lanzar	El mensaje de lanzamiento del sub-agente define su contexto completo — cuanto más preciso, menos tokens gasta en inferir qué hacer	Variable, significativo en tareas repetitivas
Límite de reintentos	Definir en <code>AGENTS.md</code> un máximo de iteraciones por tarea (ej: máximo 3 revisiones)	Evita loops costosos por ambigüedad
Skills compartidos en lugar de instrucciones repetidas	Poner lógica recurrente en skills en <code>~/.openclaw/skills</code> — disponible para todos los agentes sin repetir tokens	Reduce contexto en cada llamada

→ **Siguiente módulo: Conexión a APIs**
Auth flows reales, manejo de errores, retries y casos concretos con Google Workspace, APIs REST y webhooks.